Книга 4

Практическое применение ИИ

Инструменты, API, агенты, RAG, автоматизация рабочих процессов и безопасная интеграция

Начинающий — Продвинутый

Библиотека ИИ · Книга 4 из 5

Содержание

1. 1. Карта инструментов: что существует и для чего
2. 2. Работа с API языковых моделей
3. 3. RAG: поиск + генерация
4. 4. ИИ-агенты: автономные действия
5. 5. Автоматизация рабочих процессов
6. 6. ИИ для контента: текст, изображения, видео
7. 7. ИИ в разработке ПО
8. 8. ИИ в бизнесе и аналитике
9. 9. Безопасная интеграция: что проверять
10. 10. Практикум: строим RAG-систему

1

Карта инструментов

Что существует и для чего

Современный ИИ — это не одна программа, а целая экосистема инструментов. Важно понимать, какой инструмент для какой задачи.

Категория	Примеры	Для чего
Чат-модели (LLM)	ChatGPT, Claude, Gemini, Copilot	Текст, диалоги, анализ, код
Генерация изображений	Midjourney, DALL-E, Stable Diffusion	Иллюстрации, дизайн, арт
Генерация видео	Sora, Runway, Pika	Видеоконтент, анимация
Генерация аудио	ElevenLabs, Suno	Озвучка, музыка
Транскрипция	Whisper, Otter.ai	Речь → текст, субтитры
Перевод	DeepL, Google Translate	Перевод с сохранением контекста
Поиск + ИИ	Perplexity, Bing Chat	Поиск с генеративным ответом
Программирование	GitHub Copilot, Cursor, Codeium	Автодополнение, рефакторинг
Специализированные	AlphaFold, RoseTTAFold	Наука, медицина, биология

Главный принцип выбора

Не ищите «лучший ИИ вообще». Ищите лучший инструмент для конкретной задачи. Для перевода лучше DeepL, для написания кода — Copilot, для рассуждений — Claude, для изображений — Midjourney.

Ключевые выводы

- ИИ-инструменты делятся на категории по типу задачи
- Нет «лучшего ИИ вообще» — есть лучший для конкретной задачи
- Многие задачи решаются связкой нескольких инструментов

2

Работа с API языковых моделей

Как подключить ИИ к своему приложению

API позволяет отправлять промпты к языковой модели программно и получать ответ в коде. Это основа любой ИИ-интеграции.

Пример: вызов OpenAI API (Python)

```
from openai import OpenAI

client = OpenAI(api_key="sk-...")

response = client.chat.completions.create(
model="gpt-4o",
messages=[
{"role": "system", "content": "Ты — помощник-переводчик."},
{"role": "user", "content": "Переведи на английский: Привет, мир!"},
],
temperature=0.7,
max_tokens=100
)

print(response.choices[0].message.content)
# → "Hello, world!"
```

Ключевые параметры API

Параметр	Что делает	Типичные значения
model	Какую модель использовать	gpt-4o, claude-3.5-sonnet, gemini-pro
temperature	Креативность (0 = точно, 1 = разнообразно)	0.0 — 1.0
max_tokens	Максимум токенов в ответе	100 — 4096
top_p	Ограничение выбора токенов (nucleus sampling)	0.9 — 1.0
stream	Ответ по частям (streaming)	True / False
messages	История диалога с ролями	system / user / assistant

Streaming — зачем и как

Streaming позволяет получать ответ по частям, как только модель генерирует токены. Это даёт эффект «печатающего» ответа и снижает воспринимаемое время ожидания.

```
stream = client.chat.completions.create(
model="gpt-4o",
messages=[{"role": "user", "content": "Расскажи сказку"}],
stream=True,
)

for chunk in stream:
if chunk.choices[0].delta.content:
print(chunk.choices[0].delta.content, end="")
```

Стоимость

API тарифицируется по токенам: отдельно входные (prompt tokens) и выходные (completion tokens). Входные дешевле. Контекст в сообщениях тоже считается. Длинный диалог может стоить дороже, чем вы ожидаете.

Ключевые выводы

- API = программный доступ к языковой модели
- Ключевые параметры: model, temperature, max_tokens, messages
- Streaming даёт ответ по частям — лучше UX
- Стоимость зависит от количества токенов (вход + выход)

3 RAG: поиск + генерация

Retrieval-Augmented Generation

RAG — это техника, при которой модель перед ответом ищет информацию в вашей базе знаний. Это решает две проблемы: галлюцинации и устаревшие знания модели.

Как работает RAG

1. Документы разбиваются на куски (chunks) по 200-1000 токенов
2. Каждый кусок превращается в embedding-вектор и сохраняется в векторной базе данных
3. При запросе пользователя: запрос тоже превращается в вектор
4. Векторная база находит ближайшие (cosine similarity) куски документов
5. Найденные куски + исходный запрос отправляются в LLM
6. LLM генерирует ответ на основе найденного контекста

```
Схема RAG:

Документы → chunks → embeddings → векторная БД
↑
Запрос → embedding → поиск ближайших → context
↓
Промпт = [context + запрос] → LLM → ответ с цитатами
```

Компоненты RAG-системы

Компонент	Примеры	Задача
Embedding-модель	text-embedding-3-small, BGE, E5	Текст → вектор
Векторная БД	Pinecone, Weaviate, Qdrant, pgvector	Хранение и поиск по сходству
LLM	GPT-4o, Claude, LLaMA	Генерация ответа по контексту
Фреймворк	LangChain, LlamaIndex	Оркестрация пайплайна

□ Когда RAG лучше fine-tuning

Если данные часто меняются (документация, база знаний, новости) — RAG лучше. Fine-tuning нужен, когда меняется стиль или поведение модели, а не факты. Часто их комбинируют.

Ключевые выводы

- RAG = поиск по базе знаний + генерация ответа
- Решает галлюцинации и устаревание знаний
- Компоненты: embedding-модель, векторная БД, LLM, фреймворк

4 ИИ-агенты: автономные действия

Когда ИИ не только отвечает, но и действует

ИИ-агент — это система, в которой языковая модель не просто отвечает на вопросы, а планирует шаги, вызывает инструменты, анализирует результаты и принимает решения.

Архитектура агента

```
Цель пользователя
↓
LLM планирует шаги (Planner)
↓
┌──→ Выбрать инструмент
│ ↓
│ Вызвать инструмент (search, code, API, DB)
│ ↓
│ Получить результат
│ ↓
│ LLM анализирует: цель достигнута?
│ ↓
└── Нет → следующий шаг
Да → финальный ответ
```

Типы инструментов (tools)

- Поиск в интернете (web search, API новостей)
- Выполнение кода (Python sandbox, REPL)
- Запросы к базе данных (SQL, API)
- Чтение/запись файлов
- Отправка сообщений (email, Slack, Telegram)
- Вызов внешних API (CRM, платёжные системы, аналитика)
- Использование других ИИ-моделей (генерация изображений, транскрипция)

Фреймворки для агентов

Фреймворк	Особенность
LangChain Agents	Гибкий, множество готовых инструментов
LangGraph	Графы состояний для сложных агентов
CrewAI	Мульти-агентные системы с ролями
AutoGen (Microsoft)	Многоагентные диалоги
OpenAI Assistants API	Готовое API от OpenAI с инструментами

Безопасность агентов

Агенты с доступом к реальным инструментам (код, БД, email) — это реальная власть. Обязательно: ограничение прав, подтверждение критичных действий человеком (human-in-the-loop), логирование, пределы бюджета и времени.

Ключевые выводы

- Агент = LLM + инструменты + цикл «план → действие → анализ»
- Инструменты: поиск, код, БД, API, коммуникации
- Безопасность: ограничение прав, human-in-the-loop, логи

5

Автоматизация рабочих процессов

Где ИИ экономит время каждый день

ИИ лучше всего работает там, где есть повторяющиеся задачи с текстом, данными или решениями. Вот конкретные сценарии.

Сценарий: Обработка почты

Сортировка входящих по категориям, черновики ответов, извлечение ключевых действий из писем.

Сценарий: Суммаризация документов

Длинные отчёты, договоры, совещания → краткое резюме с ключевыми пунктами и решениями.

Сценарий: Создание контента

Посты для соцсетей, описания товаров, email-рассылки, статьи — черновик за минуты, правка за часы.

Сценарий: Перевод и локализация

Не просто перевод, а адаптация под культуру, тон и аудиторию целевого рынка.

Сценарий: Анализ отзывов

Автоматическая классификация по темам, тональности, выявление повторяющихся проблем.

Сценарий: Подготовка данных

Очистка, форматирование, категоризация неструктурированных данных в таблицы и JSON.

Сценарий: Код-ревью

Автоматическая проверка PR: баги, стиль, безопасность, предложения улучшений.

> **Принцип внедрения**
>
> Начните с одной повторяющейся задачи. Измерьте время «до» и «после». Если экономия значимая — масштабируйте. Не пытайтесь автоматизировать всё сразу.

Ключевые выводы

- ИИ лучше всего для повторяющихся текстовых задач
- Начните с одной задачи и измерьте эффект
- Суммаризация, контент, анализ, перевод — топ-сценарии

6

ИИ для контента: текст, изображения, видео

Что умеют генеративные модели

Текст

Задача	Инструмент	Результат
Статьи, посты	ChatGPT, Claude, Jasper	Черновик за минуты
Рекламные тексты	Copy.ai, Writesonic	Заголовки, CTA, лендинги
SEO-контент	Surfer SEO + AI	Оптимизированные статьи
Тех. документация	Claude, GPT-4 + RAG	Документация по коду/API

Изображения

Инструмент	Сильная сторона
Midjourney	Художественность, эстетика, стили
DALL-E 3	Точность по промпту, интеграция с ChatGPT
Stable Diffusion	Open-source, локальный запуск, контроль
Adobe Firefly	Коммерческая безопасность, интеграция с Adobe

Промпт для изображения

```
Midjourney:
/imagine prompt: editorial photography, modern minimalist office,
soft natural light, shallow depth of field, 35mm, color grading
teal and orange, high detail, photorealistic --ar 16:9 --v 6
```

Видео и аудио

- Sora / Runway — генерация видео по описанию, до 60 секунд
- Synthesia — видео с ИИ-аватаром (презентации, обучение)
- ElevenLabs — реалистичная озвучка текста, клонирование голоса
- Suno / Udio — генерация музыки с вокалом по описанию
- Whisper — транскрипция речи в текст с высокой точностью

> ⚖ **Авторское право**
>
> Контент, созданный ИИ, может иметь ограничения по авторскому праву в разных юрисдикциях. Проверяйте условия использования платформы и не используйте чужие бренды/стили художников без разрешения.

Ключевые выводы

• Текст: ChatGPT, Claude для черновиков; специализированные — для маркетинга

• Изображения: Midjourney (арт), DALL-E (точность), SD (контроль)

• Видео/аудио: Sora, ElevenLabs, Suno, Whisper

• Проверяйте авторские права и условия использования

7

ИИ в разработке ПО

Copilot, Cursor, ИИ-ассистенты в IDE

ИИ изменил процесс написания кода. Автодополнение, генерация функций, рефакторинг, тесты, документация — всё это ускоряется в разы.

Инструмент	Что делает
GitHub Copilot	Автодополнение кода, генерация функций, коммит-месседжи
Cursor	IDE с ИИ: чат с кодовой базой, мультифайловое редактирование
Claude / GPT-4	Сложные задачи: архитектура, отладка, объяснение кода
Codeium / Tabnine	Бесплатные альтернативы автодополнения
v0 / Bolt	Генерация UI-компонентов и целых приложений по описанию

Что ИИ делает хорошо в коде

- Автодополнение — продолжает код, который вы начали писать
- Генерация boilerplate — скелет функции, класса, API-эндпоинта
- Тесты — генерация юнит-тестов по реализации
- Документация — docstrings, README, комментарии
- Рефакторинг — переименование, упрощение, разбиение функций
- Отладка — анализ ошибки, предложения исправления
- Перевод между языками — Python → Go, JavaScript → TypeScript

⚠ **Где ИИ ошибается в коде**

Сложная бизнес-логика, нюансы архитектуры, устаревшие API, баги в зависимостях. ИИ может предложить код, который выглядит правильно, но не работает. Всегда проверяйте и тестируйте.

Ключевые выводы

- Copilot/Cursor ускоряют рутину: boilerplate, тесты, документация
- ИИ хорошо генерирует код по описанию, но сложную логику — проверяйте
- Используйте ИИ как ассистента, а не замену понимания

8 ИИ в бизнесе и аналитике

Где ИИ создаёт реальную ценность

В бизнесе ИИ применяется не для «инноваций ради инноваций», а для конкретных задач: снижение затрат, рост выручки, ускорение процессов.

Область	Применение ИИ	Эффект
Поддержка клиентов	Чат-боты + RAG по базе знаний	Снижение нагрузки на 40-60%
Продажи	Оценка лидов, персонализация предложений	Рост конверсии 15-30%
Маркетинг	Сегментация, A/B-тесты, генерация контента	Экономия времени на создание кампаний
Аналитика	Анализ отзывов, прогнозы спроса, аномалии	Быстрее insights, меньше ручной работы
HR	Сортировка резюме, онбординг, обучение	Скорость найма, персонализация обучения
Финансы	Обнаружение мошенничества, скоринг, отчёты	Снижение потерь, автоматизация отчётности
Логистика	Оптимизация маршрутов, прогноз запасов	Снижение затрат на транспортировку

Аналитика данных с ИИ

Современные инструменты (ChatGPT Advanced Data Analysis, Julius AI, Hex Magic) позволяют анализировать данные без кода: загрузить CSV, задать вопрос на естественном языке, получить график и выводы.

```
Пример запроса к данным:

"Проанализируй продажи за последний квартал. Найди топ-5 продуктов по выручке, покажи динамику по неделям и выдели аномалии."
```

Ключевые выводы

- ИИ в бизнесе = снижение затрат + рост выручки
- Топ-области: поддержка, продажи, маркетинг, аналитика
- Data analysis без кода — реальность с современными инструментами

9

Безопасная интеграция: что проверять

Чек-лист для внедрения ИИ в продакшен

ИИ в продакшене отличается от демо. Вот что нужно проверить перед запуском и контролировать после.

До запуска

1. Валидация входа: проверка размера, формата, содержания пользовательского ввода
2. Ограничение rate limit: защита от перебора и злоупотреблений
3. Бюджет на API: оценка стоимости и установка лимитов (дневных, месячных)
4. Fallback: что происходит, если API недоступен или отвечает с ошибкой
5. Приватность данных: не отправляются ли чувствительные данные в облачный API
6. Тестирование на галлюцинации: проверка типичных запросов и edge cases
7. Инструкции безопасности в system prompt: что модель не должна делать

В продакшене

1. Логирование всех запросов и ответов (без чувствительных данных)
2. Мониторинг стоимости, задержки, ошибок
3. Оповещения при всплеске ошибок или стоимости
4. Регулярная проверка качества ответов (выборка + оценка)
5. Обновление моделей: тестировать перед переключением
6. Сбор фидбека от пользователей (thumbs up/down, комментарии)

Промпт-инъекции — главная угроза

Злоумышленник может встроить инструкции в пользовательский ввод, чтобы заставить модель проигнорировать системные правила. Никогда не вставляйте пользовательский текст в system prompt без санитизации. Разделяйте инструкции и данные.

Ключевые выводы

- Валидация входа, rate limit, бюджет, fallback — обязательны
- Логирование и мониторинг — в продакшене
- Промпт-инъекции — главная угроза, разделяйте инструкции и данные

10 Практикум: строим RAG-систему

Минимальный RAG на Python за 50 строк

Минимальная RAG-система: читаем документы, создаём embeddings, ищем по запросу, генерируем ответ.

```
import openai
import numpy as np

client = openai.OpenAI(api_key="sk-...")

# 1. Документы
docs = [
"GPT-4 выпущен в марте 2023 года, имеет 128K контекст.",
"Claude 3 Opus от Anthropic выпущен в марте 2024.",
"LLaMA 2 — open-source модель от Meta, 70B параметров.",
]

# 2. Embeddings
def embed(text):
r = client.embeddings.create(
model="text-embedding-3-small", input=text)
return np.array(r.data[0].embedding)

doc_vectors = [embed(d) for d in docs]

# 3. Поиск
def search(query, top_k=2):
q_vec = embed(query)
scores = [np.dot(q_vec, dv) for dv in doc_vectors]
idx = np.argsort(scores)[-top_k:][::-1]
return [docs[i] for i in idx]

# 4. Ответ
def answer(query):
context = "\n".join(search(query))
r = client.chat.completions.create(
model="gpt-4o",
messages=[
{"role": "system",
"content": f"Ответь по контексту. Если нет данных — скажи.\n\n{context}"},
{"role": "user", "content": query},
])
return r.choices[0].message.content

print(answer("Какая модель open-source?"))
# → "LLaMA 2 — open-source модель от Meta, 70B параметров."
```

Что улучшить для продакшена

- Векторная БД вместо списка (Pinecone, Qdrant, pgvector)
- Разбиение документов на chunks (не целые документы)
- Гибридный поиск: векторный + ключевое слово (BM25)
- Переранжирование результатов (cross-encoder)
- Кэширование embeddings и ответов
- Цитирование источников в ответе

- Оценка качества: RAGAS или ручная выборка

Конец Книги 4. Следующая — Книга 5: Этика, безопасность и ограничения ИИ.